

1. Syntax Rules and Critical Chat Corrections

- **The Shebang & Execution:** Every script must start with `#!/bin/bash` or `#!/bin/sh`. You must grant execution rights using `% chmod +x script` and run it with `% ./script`.
- **The Spacing Rule:** Square brackets require spaces on the inside: `[ $val -eq 0 ]`.
- **Variable Assignments:** Do not use spaces around the equals sign (e.g., `count=0`).
- **Math vs. Strings:** Writing `count=$count+1` performs string concatenation, resulting in an infinite loop. You must use the `let` keyword for arithmetic: `let count=$count+1`.
- **Quoting Variables:** Always wrap variables in double quotes (e.g., `"$Net" -eq "0"`) inside test conditions. This prevents the script from crashing if the variable is empty or contains spaces.
- **The Semicolon (;) & then:** If you place `then` on the same line as the `if` condition, you must separate them with a semicolon: `if [ condition ]; then`.
- **The -n Flag's Dual Purpose:** When used as `echo -n`, it prevents outputting a new line. When used inside brackets as `[ -n "$string" ]`, it evaluates true if the string length is greater than zero.

2. Shell Variables and Operators Cheat Sheet

Input & Special Variables The `read` command captures user input, and the last variable listed absorbs the rest of the line. The `set` command manually assigns values to positional parameters directly in the terminal.

Variable	Definition
\$0	Name of the current shell script.
\$1-\$9	Positional parameters 1 through 9.
\$#	The total number of positional parameters.
\$*	All positional parameters merged into one single string.
\$@	All positional parameters treated as a set of individual strings.
\$?	Return status of the most recently executed command.
\$\$	Process ID of the current process.

\$REPLY	Built-in variable storing the raw number typed during a select menu.
---------	--

Logic, Relational, and File Operators Use double brackets [[]] when combining conditions with && (AND) or || (OR). The ! symbol acts as a logical NOT operator.

File Flags	Meaning	Relational	Meaning
-d file	True if directory.	-gt / -ge	Greater than / Greater or equal.
-f file	True if ordinary file.	-lt / -le	Less than / Less or equal.
-r file	True if readable.	-eq / -ne	Equal / Not equal.
-w file	True if writable.	= or ==	String equal.
-x file	True if executable.	!=	String not equal.
-s file	True if length is non-zero.	-z str	String length is zero.

3. Control Structures and Loop Templates

If-Elif-Else Statement

```
Bash
if [ condition ]; then
    statements
elif [ condition ]; then
    statement
else
    statements
fi
```

(Reference: Slide 13)

Case Statement (Multi-Choice) Requires double semicolons ;; at the end of each

command list.

```
Bash
case word in
  pattern1) command-list1 ;;
  pattern2) command-list2 ;;
esac
```

(Reference: Slide 24)

Loops (While, Until, For, Select)

- while [expression]: Executes commands as long as the expression evaluates to true.
- until [expression]: Executes commands as long as the expression evaluates to false.
- for variable in argument-list: Iterates sequentially over a provided list of words.
- select WORD in LIST: Automatically constructs a numbered menu and loops until the user presses Ctrl+D.
- break: Instantly terminates a loop and transfers control to the statement after done.
- continue: Skips the rest of the current iteration and jumps to the next loop iteration.

```
Bash
# General Loop Structure (While/Until/For/Select)
loop-command [ condition / list
do
  commands
done
```

(Reference: Slides 31, 35, 38, 42)

4. Functions, Signals, and Debugging

Functions Functions must be defined before they are called. Variables inside functions are globally accessible unless restricted using the local keyword.

```

Bash
function name () {
    local variable_name="value"
    statements
}

```

(Reference: Slides 52, 58)

Signals and Traps A signal is a system message sent to a process (like a keyboard interrupt), and a signal handler dictates the process's reaction. The trap command intercepts these signals to run custom code instead of crashing. The sleep command pauses script execution for a specified number of seconds to prevent processor overload in infinite loops.

- **Trap Template:** trap 'handler commands' signals.
- **Restoring Default Handler:** trap 2 (running without commands removes the custom trap).

Signal Number	Name	Description / Behavior
1	SIGHUP	Hangup signal.
2	SIGINT	Interrupt signal triggered by pressing Ctrl+C.
9	SIGKILL	The absolute force-quit kill command; it cannot be blocked or trapped.
15	SIGTERM	Terminate signal; the default behavior when running a plain kill command.

Debugging Use the set command to trace execution line-by-line.

- set -xv: Turns on debugging (prints read lines and arguments).
- set +xv: Turns off debugging.